

XRPL Validator — Daily Engineering Report • 2026-08-14

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **9 commits** (xrpl-validator 9)

Engineering changes

- **fix(xrpl-ledger): measure the REVERSE pass's consumption precisely too** (45a7092 , xrpl-validator)
44c20d9 fixed the forward carry and I did not notice the same defect one level up. `measure_hop` worked out how much a trial consumed by differencing a GRANTED balance — 16 significant digits TOTAL, so the grant itself eats the low end of the answer — while `cross_engine_to` was already RETURNING the precise remaining input and the code threw it away with `let (rw, _, _) = ...`.
- **fix(xrpl-ledger): the precise measurement must account for the fee the walk debits** (1f31546 , xrpl-validator)
A defect in 45a7092. Its precise path is `granted - rem_in`, but `rem_in` tracks only the NET the walk moved — the input transfer fee is debited SEPARATELY by `line_adjust`. So on a fee-bearing hop `precise` is the net while the differenced balance `d` is the gross, the two disagree by the WHOLE FEE (1e-3 for a 1.001 issuer, a thousand times the 1e-9 agreement threshold), and the guard rejected the precise value ...
- **fix(xrpl-ledger): ask the walk what a strand spent, don't difference the balance** (8d8d2e6 , xrpl-validator)
Third instance of the defect 44c20d9 fixed for a hop's carry and 45a7092 for the reverse pass — and exactly the `signed_add(before, !after)` shape 45a7092's note said to go looking for. `strand_pass` reported its input consumption by differencing the sender's spend-leg balance, and a balance holds 16 significant digits TOTAL, so whatever the sender already holds of the spend currency eats the low end of the ...
- **feat(xrpl-ledger): one AMM consumption per payment-engine iteration** (4566c4e , xrpl-validator) "At any payment engine iteration, AMM offer can only be consumed once" (BookStep.cpp:818). When a payment wants more pool liquidity than one AMM offer gives, rippled ENDS the pass and `flow()` re-enters the strand for the next one — and every iteration writes the maker's residual, so N iterations are N roundings. We took Fibonacci slices inside a single pass and rounded once.
- **fix(xrpl-ledger): the bridge's direct head prices a partial fill at the filed rate** (7f8286b , xrpl-validator) The rule e2a9c99 gave the two bridge legs, never given to the third fill site in the same function. An offer carries two rates: the one it is FILED under (low 64 bits of `BookDirectory`, a 16-digit mantissa fixed at creation) and its current `TakerPays/TakerGets`, which drifts every time a residual is re-rounded. rippled prices a PARTIAL fill at the filed rate and a FULL one at the amounts.
- **fix(xrpl-ledger): limitOut takes the STRICT canonicaliser when the drops decide** (ff1e752 , xrpl-validator)
rippled's own comment on `T0ffer::limitOut` (Offer.h:200): "It turns out that the `ceil_out` implementation has some slop in it, which `ceil_out_strict` removes." So an out-limited fill goes `ceilOutStrict -> mulRoundStrict`

-> `canonicalizeRoundStrict`, not `mulRound`. After dividing down to offset -1 the two differ by one term (STAmount.cpp:1477 vs :1516): `canonicalizeRound` value += 9; `canonicalizeRoundStrict` value ...

- **fix(xrpl-ledger): rippleSend moves the sender's line ONCE, with the gross** (`bc756dd` , `xrpl-validator`)
rippled's `rippleSend` (View.cpp) is one credit per party: `saActual = saAmount + saTransitFee`;
`rippleCredit(issuer, receiver, saAmount); // receiver gets the NET` `rippleCredit(sender, issuer, saActual); // sender`
pays the GROSS
- **fix(xrpl-ledger): every payment strand runs ONE PASS — corpus byte parity 0 hits** (`950e63e` , `xrpl-validator`)
The third and last part of the iteration model, and the SAME `if multi` gate the rounds carried in `4566c4e` . A pass ends when an offer is CONSUMED and `flow()` re-enters the strand; `single_pass` already implements that, but it was switched on only for multi-strand payments, so a lone strand walked the whole book in one call and MERGED what rippled does in separate iterations.
- **fix(xrpl-ledger): NFTokenAcceptOffer weighs the buyer's funds — and the probe lets it see them** (`9c0d845` , `xrpl-validator`)
9 of the 13 divergences in the 2026-08-14 fresh sweep were one cluster:
`our_ter=tecOBJECT_NOT_FOUND` against `net_ter=tecINSUFFICIENT_FUNDS` , 1 mutation each side. TWO defects stacked, and fixing either alone measures WORSE than fixing neither.

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **PayChannel Merkle Batch Claims** (gaming) Add an `lsfMultiClaim` flag and `sfClaimantRoot` (Hash256 Merkle root over {Destination, CumulativeAmount} leaves) to PayChannel, plus a new PayChannelBat
- **Commit-Reveal Offers Against Front-Running** (dex) Add two transaction types: OfferCommit, creating an owner-reserved `ltOFFER_COMMIT` object holding `sfCommitHash` (Hash256 = sha512Half(TakerPays||TakerGe
- **Credentialed Sponsor-Onboarding for MPTs** (tokens) Extend `MPTokenAuthorize` with optional `sfSponsor` (AccountID) and `sfSponsorCredentialType` SFields: when present, preclaim looks up an existing Credentia

Appendix — commits

Time	Repo	Commit	Subject
00:06	xrpl-validator	45a7092	fix(xrpl-ledger): measure the REVERSE pass's consumption precisely too
10:26	xrpl-validator	1f31546	fix(xrpl-ledger): the precise measurement must account for the fee the walk debits
12:31	xrpl-validator	8d8d2e6	fix(xrpl-ledger): ask the walk what a strand spent, don't difference the balance
13:30	xrpl-validator	4566c4e	feat(xrpl-ledger): one AMM consumption per payment-engine iteration
15:34	xrpl-validator	7f8286b	fix(xrpl-ledger): the bridge's direct head prices a partial fill at the filed rate
16:37	xrpl-validator	ff1e752	fix(xrpl-ledger): limitOut takes the STRICT canonicaliser when the drops decide
17:31	xrpl-validator	bc756dd	fix(xrpl-ledger): rippleSend moves the sender's line ONCE, with the gross
18:34	xrpl-validator	950e63e	fix(xrpl-ledger): every payment strand runs ONE PASS — corpus byte parity 0 hits
21:40	xrpl-validator	9c0d845	fix(xrpl-ledger): NFTokenAcceptOffer weighs the buyer's funds — and the probe lets it see them